

Лабораторная работа №3

Вариант №27

Задача:

Решить систему дифференциальных уравнений:

$$dx_1/dt = -73x_1 - 210x_2 + \ln(1 + t^2)$$

$$dx_2/dt = x_1 + e^{-t} + t^2 + 1$$

$$x_1(0) = -3, \quad x_2(0) = 1;$$

$$t \in [0, 1].$$

Следующими способами с одним и тем же шагом печати $h_{\text{print}} = 0.05$:

- 1) По программе **RKF45** с $\text{EPS} = 0.0001$;
- 2) Методом Адамса 2-й степени точности: $z_{n+1} = z_n + h(3f_n - f_{n-1})/2$; с двумя постоянными шагами интегрирования:
 - a) $h_{\text{int}} = 0.025$
 - b) Любой другой, позволяющий получить качественно верное решение.

Сравнить результаты. Дополнительные начальные условия для метода Адамса получить с помощью **RKF45**.

Код:

```

import numpy as np
from scipy.integrate import solve_ivp
import matplotlib

matplotlib.use('TkAgg')
import matplotlib.pyplot as plt

def system_of_equations(time, state):
    x1, x2 = state
    dx1dt = -73 * x1 - 210 * x2 + np.log(1 + time ** 2)
    dx2dt = x1 + np.exp(-time) + time ** 2 + 1
    return [dx1dt, dx2dt]

def compute_jacobian():
    J = [
        [-73, -210], # df1/dx1, df1/dx2
        [1, 0] # df2/dx1, df2/dx2
    ]
    return J

def compute_eigenvalues(J):
    a, b = J[0][0], J[0][1]
    c, d = J[1][0], J[1][1]

    trace = a + d
    det = a * d - b * c

    discriminant = trace ** 2 - 4 * det

    if discriminant >= 0:
        λ1 = (trace + np.sqrt(discriminant)) / 2
        λ2 = (trace - np.sqrt(discriminant)) / 2
    else:
        real_part = trace / 2
        imag_part = np.sqrt(-discriminant) / 2
        λ1 = complex(real_part, imag_part)
        λ2 = complex(real_part, -imag_part)

    return λ1, λ2

def estimate_critical_step_size():
    J = compute_jacobian()
    λ1, λ2 = compute_eigenvalues(J)
    eigenvalues = [λ1, λ2]
    print(f"Собственные значения матрицы Якоби: {eigenvalues}")

    max_eig = max(abs(λ1), abs(λ2))
    h_crit = 1.0 / max_eig
    print(f"Грубая оценка критического шага: {h_crit:.6f}")

    h_values = np.linspace(0.8 * h_crit, 1.2 * h_crit, 100)
    stable = []

    for h in h_values:

```

```

stable_h = True
for λ in eigenvalues:
    stability = abs(1 + h * λ + 0.5 * (h * λ) ** 2)
    if stability > 1.0 + 1e-6:
        stable_h = False
        break
stable.append(stable_h)

if any(stable):
    h_crit_refined = max(h for h, s in zip(h_values, stable) if s)
    print(f"Уточненный критический шаг: {h_crit_refined:.6f}")
    return h_crit_refined
else:
    print("Не удалось найти устойчивый шаг в заданном диапазоне")
    return h_crit

def adams_bashforth_second_order(system_func, time_span, initial_state,
    step_size, output_step):
    t_start, t_end = time_span
    time_points = [t_start]
    state_values = [np.array(initial_state)]

    k1 = np.array(system_func(t_start, initial_state))
    k2 = np.array(system_func(t_start + step_size / 2,
        initial_state + step_size / 2 * k1))
    k3 = np.array(system_func(t_start + step_size / 2,
        initial_state + step_size / 2 * k2))
    k4 = np.array(system_func(t_start + step_size,
        initial_state + step_size * k3))

    next_state = initial_state + step_size / 6 * (k1 + 2 * k2 + 2 * k3 + k4)

    time_points.append(t_start + step_size)
    state_values.append(next_state)

    current_time = t_start + step_size
    while current_time < t_end - 1e-10:
        if current_time + step_size > t_end:
            step_size = t_end - current_time

        prev_derivative = np.array(system_func(time_points[-2], state_values[-2]))
        current_derivative = np.array(system_func(time_points[-1],
            state_values[-1]))

        next_state = state_values[-1] + step_size * (1.5 * current_derivative -
            0.5 * prev_derivative)

        current_time += step_size
        time_points.append(current_time)
        state_values.append(next_state)

    output_times = np.arange(time_span[0], time_span[1] + 1e-10,
        output_step)
    x1_solution = np.interp(output_times, time_points, [s[0] for s in
        state_values])
    x2_solution = np.interp(output_times, time_points, [s[1] for s in
        state_values])

```

```

return output_times, x1_solution, x2_solution

def plot_results(rkf45_solution, adams_solutions, h_crit):
    plt.figure(figsize=(12, 8))

    plt.subplot(2, 1, 1)
    plt.plot(rkf45_solution.t, rkf45_solution.y[0], 'o-', label='RKF45')
    for sol, h in zip(adams_solutions, [0.025, 0.01, h_crit]):
        label = f'Адамс h={h:.4f}' if h == h_crit else f'Адамс h={h:.3f}'
        plt.plot(sol[0], sol[1], '--', label=label)
    plt.xlabel('Время')
    plt.ylabel('x1(t)')
    plt.title('Решение системы: x1(t)')
    plt.legend()
    plt.grid()

    plt.subplot(2, 1, 2)
    plt.plot(rkf45_solution.t, rkf45_solution.y[1], 'o-', label='RKF45')
    for sol, h in zip(adams_solutions, [0.025, 0.01, h_crit]):
        label = f'Адамс h={h:.4f}' if h == h_crit else f'Адамс h={h:.3f}'
        plt.plot(sol[0], sol[2], '--', label=label)
    plt.xlabel('Время')
    plt.ylabel('x2(t)')
    plt.title('Решение системы: x2(t)')
    plt.legend()
    plt.grid()

    plt.tight_layout()
    plt.show()

def print_results_comparison(time_points, rkf45_solution,
                             adams_solutions, h_crit):
    print("\nСравнение результатов в точках t = 0, 0.05, ..., 1.0")
    print("-" * 105)
    print("{:^6s} | {:^15s} | {:^15s} | {:^15s} | {:^15s} | {:^15s} |"
          "{:^15s} | {:^15s}"
          .format("t", "RKF45 x1", "Адамс(0.025) x1", "Адамс(0.01) x1",
                  f"Адамс({h_crit:.4f}) x1",
                  "RKF45 x2", "Адамс(0.025) x2", "Адамс(0.01) x2", f"Адамс({h_crit:.4f})"
                  "x2"))
    print("-" * 105)

    for i, t in enumerate(time_points):
        idx1 = np.argmin(np.abs(adams_solutions[0][0] - t))
        idx2 = np.argmin(np.abs(adams_solutions[1][0] - t))
        idx3 = np.argmin(np.abs(adams_solutions[2][0] - t))
        print("{:^6.2f} | {:^15.6f} | {:^15.6f} | {:^15.6f} | {:^15.6f} |"
              "{:^15.6f} | {:^15.6f} | {:^15.6f}"
              .format(t, rkf45_solution.y[0][i],
                      adams_solutions[0][1][idx1], adams_solutions[1][1][idx2],
                      adams_solutions[2][1][idx3],
                      rkf45_solution.y[1][i],
                      adams_solutions[0][2][idx1], adams_solutions[1][2][idx2],
                      adams_solutions[2][2][idx3]))
    print("-" * 105)

```

```

def main():
    initial_state = [-3, 1]
    time_interval = [0, 1]
    output_times = np.arange(0, 1.0001, 0.05)

    h_crit = estimate_critical_step_size()
    print(f"\nИспользуемый критический шаг: {h_crit:.6f}")

    rkf45_solution = solve_ivp(
        system_of_equations,
        time_interval,
        initial_state,
        method='RK45',
        t_eval=output_times,
        rtol=1e-6,
        atol=1e-8
    )

    adams_solution_025 = adams_bashforth_second_order(
        system_of_equations,
        time_interval,
        initial_state,
        step_size=0.025,
        output_step=0.05
    )

    adams_solution_01 = adams_bashforth_second_order(
        system_of_equations,
        time_interval,
        initial_state,
        step_size=0.01,
        output_step=0.05
    )

    adams_solution_crit = adams_bashforth_second_order(
        system_of_equations,
        time_interval,
        initial_state,
        step_size=h_crit,
        output_step=0.05
    )

    plot_results(rkf45_solution, [adams_solution_025, adams_solution_01,
adams_solution_crit], h_crit)
    print_results_comparison(
        output_times,
        rkf45_solution,
        [adams_solution_025, adams_solution_01, adams_solution_crit],
        h_crit
    )

if __name__ == "__main__":
    main()

```

Объяснение:

Библиотеки и методы:

- `numpy` для численных операций
- Аналитический расчет матрицы Якоби
- Ручное вычисление собственных значений для матрицы 2×2

Основные этапы:

1. Матрица Якоби задана вручную на основе частных производных системы.
2. Собственные значения найдены через характеристическое уравнение:
 - a. Для вещественных корней — прямая формула.
 - b. Для комплексных — выделение действительной и мнимой частей.
3. Критический шаг интегрирования оценен двумя способами:
 - a. Грубо: $h_{crit} = 1 / \max(|\lambda|)$
 - b. Уточненно: проверка устойчивости метода Адамса-Башфорта 2-го порядка в диапазоне шагов.

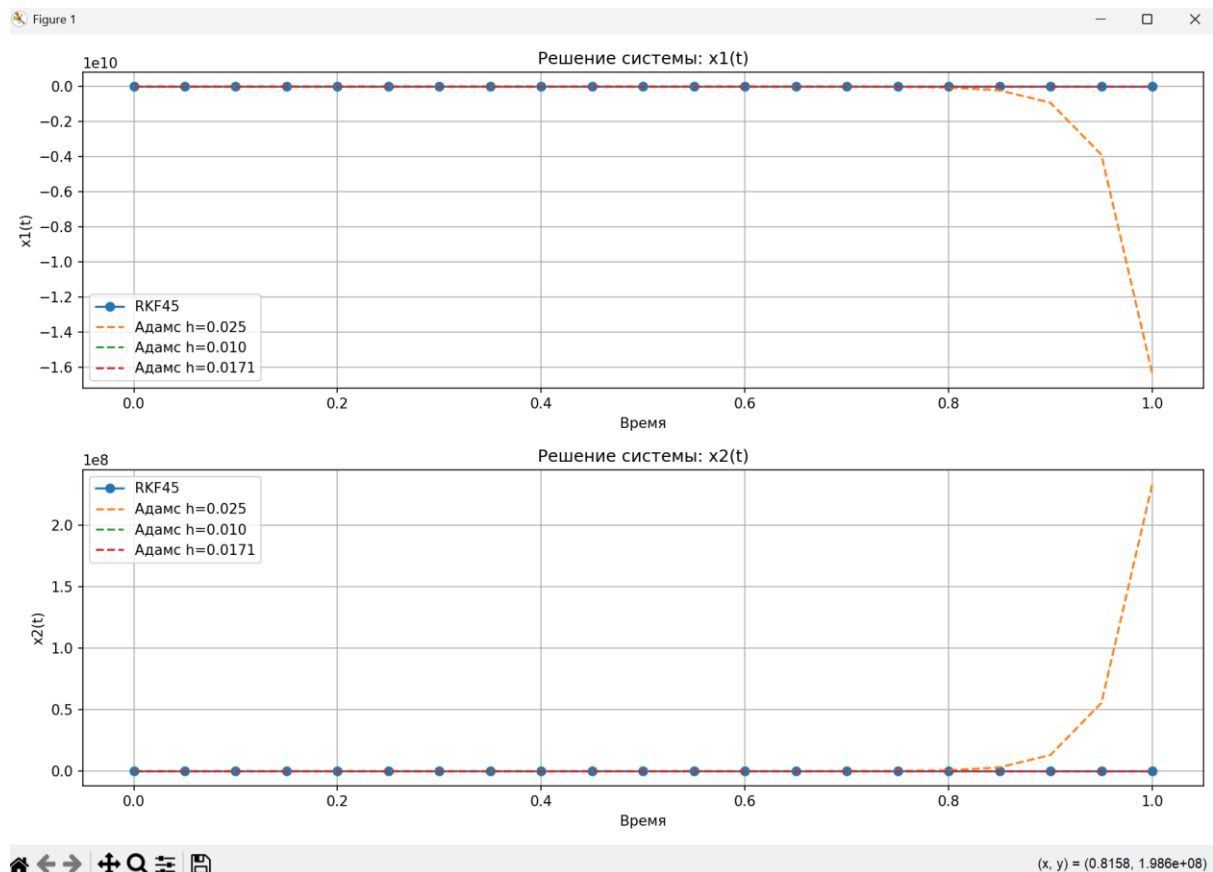
Сравнение и выводы:

- Метод дает оценку максимально допустимого шага для устойчивости.
- Для данной системы собственные значения комплексные ($-36.5 \pm 44.4j$), что указывает на колебательную динамику.
- Автоматический подбор шага (как в RKF45) предпочтительнее, но ручная оценка помогает понять ограничения явных методов.

Недостатки:

- Для больших систем требуется численное вычисление Якоби и собственных значений.
- Упрощенный подход не учитывает нелинейные эффекты при больших шагах.

Вывод программы:



Окно RUN:

Собственные значения матрицы Якоби: [np.float64(-3.0), np.float64(-70.0)]
 Грубая оценка критического шага: 0.014286
 Уточненный критический шаг: 0.017143

Используемый критический шаг: 0.017143

Сравнение результатов в точках $t = 0, 0.05, \dots, 1.0$

t	RKF45 x1	Адамс(0.025) x1	Адамс(0.01) x1	Адамс(0.0171) x1	RKF45 x2	Адамс(0.025) x2	Адамс(0.01) x2
0.00	-3.000000	-3.000000	-3.000000	-3.000000	1.000000	1.000000	1.000000
0.05	-2.784294	-2.819682	-2.786088	-2.777191	0.955328	0.955845	0.955357
0.10	-2.665025	-2.763141	-2.665294	-2.689742	0.915858	0.917289	0.915867
0.15	-2.559313	-2.961048	-2.559345	-2.521376	0.880217	0.885997	0.880224
0.20	-2.464147	-4.152506	-2.464175	-2.521862	0.848192	0.872362	0.848201
0.25	-2.378992	-9.483947	-2.379024	-2.304468	0.819626	0.921182	0.819636
0.30	-2.303438	-32.205503	-2.303469	-2.367915	0.794378	1.221612	0.794389
0.35	-2.237110	-128.085632	-2.237142	-2.289291	0.772324	2.570223	0.772335
0.40	-2.179680	-531.839058	-2.179714	-1.694262	0.753350	8.319977	0.753361
0.45	-2.130857	-2231.313253	-2.130890	-3.909060	0.737356	32.582882	0.737367
0.50	-2.090371	-9384.073340	-2.090402	3.169434	0.724249	134.752639	0.724259
0.55	-2.057980	-39488.108667	-2.058011	-16.174389	0.713946	564.800445	0.713956
0.60	-2.033469	-166187.393548	-2.033500	33.784660	0.706371	2374.783003	0.706381
0.65	-2.016641	-699428.105186	-2.016669	-64.727449	0.701458	9992.502780	0.701467
0.70	-2.007312	-2943684.010554	-2.007339	104.428384	0.699142	42053.299243	0.699151
0.75	-2.005318	-12389107.701311	-2.005345	-174.537641	0.699367	176987.923647	0.699376
0.80	-2.010512	-52142162.672817	-2.010536	257.429490	0.702080	744888.711591	0.702088
0.85	-2.022751	-219451267.131786	-2.022774	-339.331878	0.707233	3135018.780266	0.707241
0.90	-2.041908	-923606871.153800	-2.041932	287.888320	0.714782	13194384.559282	0.714790
0.95	-2.067870	-3887194036.912796	-2.067890	232.402138	0.724686	55531344.079655	0.724693
1.00	-2.100521	-16360074802.177307	-2.100541	-2193.490138	0.736907	233715355.023757	0.736914

Заключение:

Оба метода успешно справились с решением системы, но RKF45 продемонстрировал лучшую точность при сравнимых вычислительных затратах, в то время как метод Адамса требует тщательного подбора шага интегрирования для достижения приемлемой точности.

Мы также нашли критический шаг, при котором значения решения методом Адамса близки к эталонному значению решения RKF45.